

avito.tech 

ИСТОРИЯ ПАТТЕРНОВ

И правила в их основе

A Pattern Language

Towns • Buildings • Construction



Christopher Alexander

Sara Ishikawa • Murray Silverstein

WITH

Max Jacobson • Ingrid Fiksdahl-King

Shlomo Angel

Кристофер Александер

Язык шаблонов

Города • Здания • Строительство



Издательство
Студии Артемия Лебедева

Christopher Alexander
Кристофер Александр



- чивала удобный выезд на кольцевые автодороги и съезд с них, но не способствовала движению автомобилей внутри таких районов, — см. шаблоны
- с. 104, 135 [11] Районы местного транспортного обслуживания и [17] Кольцевые автодороги. Также мы предлагали разделить эти районы на муниципальные районы и микрорайоны, но с тем условием, чтобы все главные дороги проходили по границам между муниципальными районами и микрорайонами, — см.
- с. 116, 126 [13] Границы субкультур и [15] Границы микрорайонов. Теперь рассмотрим, как должны быть расположены эти дороги, чтобы обеспечить движение внутренних транспортных потоков в [11] Районах местного транспортного обслуживания, сохранив границ районов.
- с. 104

23 Параллельные дороги



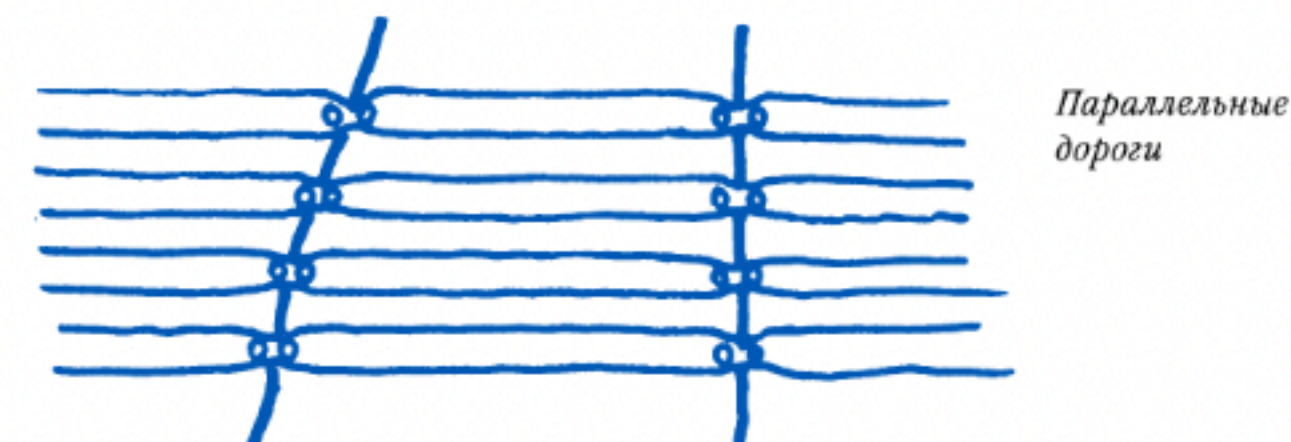
Расположение улиц в виде сети морально устарело. Города задыхаются от транспортных заторов. Автомобили могут ездить со скоростью сто километров

в час на автострадах, но средняя скорость их передвижения по городу составляет всего 15–25 километров в час.

- с. 104 Разумеется, в большинстве случаев мы хотим избавиться от автомобилей в городе, а не способствовать увеличению скорости их передвижения. Этот вопрос подробно обсуждался в шаблоне [11] Районы местного транспортного обслуживания. Но вдали от тех мест, где дети играют, а взрослые ходят пешком или ездят на велосипедах, должно все же быть несколько улиц, по которым ездят машины. Встает вопрос: как правильно спроектировать эти улицы, чтобы автомобили могли ездить быстрее и без заторов?

Оказывается, потеря скорости на улицах современных городов в основном связана с дорожными пересечениями — левыми поворотами наперекрест основному транспортному потоку и четырехсторонними пересечениями в одном уровне (Г. Ф. Ньюэлл, «Влияние левых поворотов на пропускную способность транспортных развязок». *Newell G. F. The Effect of Left Turns on the Capacity of Traffic Intersection // Quarterly of Applied Mathematics. April 1959. XVII. P. 67–76*).

Следовательно, для ускорения движения транспорта необходимо создать такую сеть главных дорог, в которой не было бы одноуровневых четырехсторонних пересечений и левых поворотов наперекрест основному потоку. Это легко осуществить за счет главных дорог с односторонним движением, идущих параллельно на расстоянии от ста до четырехсот метров друг от друга; от них будут отходить небольшие дороги местного значения, а параллельные дороги будут соединяться между собой только автострадами, пересекающими их через каждые три-пять километров.



Такая схема подробно обсуждалась в трех работах: сначала в статье Кристофера Александра «Уличный паттерн» (*Alexander C. The Pattern of Streets // AIP Journal. September 1966*); затем в критике этой публикации со стороны Д. Карсона и П. Розен-Рунге; затем в ответе на критику Кристофера Александре-

с. 601 ставление о проекте кухни и ее рабочей зоны. В паттерне [128](#) **Естественное внутреннее освещение** подчеркивалась необходимость солнечного света на кухне. Но для реализации этих более масштабных шаблонов и устройства максимально красивой и уютной кухни стоит уделить особое внимание расположению кухонного стола и окон рядом с ним.

[199](#) Кухонный стол, залитый солнцем ●



Темные мрачные кухни оказывают весьма депрессивное воздействие. В кухне требуется не меньше, а даже больше солнца, чем в других комнатах.

Посмотрите, как красиво выглядит рабочая зона на фотографии, предваряющей данный шаблон. Окна идут вдоль почти всей поверхности рабочего стола; он купается в солнечном свете, а в кухне возникает ощущение простора. Там присутствует вид наружу и все дышит покоем.



Мрачная кухня

Сравните это с мрачной кухней на другой фотографии. На рабочем столе нет естественного освещения, шкафы сгрудились в кучу; работать на такой кухне с нависающим шкафом, лицом к стене, под искусственным освещением среди бела дня — довольно недостойное занятие.

Такая мрачная кухня типична для многих тысяч подобных помещений в современных домах. Так происходит по двум причинам. Во-первых, люди часто располагают кухню с северной стороны, так как на южной стороне они хотят расположить в первую очередь жилые комнаты, а кухне отводят оставшуюся площадь дома. Во-вторых, кухню нередко воспринимают только в качестве рабочего места, предназначенного для механических операций по приготовлению пищи. Во многих квартирах «экономичные» кухни находятся в таких местах, где вообще нет естественного освещения. Но доводы, которые мы приводили в шаблоне [139](#) **Деревенская кухня** относительно необходимости превратить ее в общую семейную комнату, а не механизированный цех по приготовлению пищи, конечно, полностью меняют эту картину.

Выводы

Основная часть кухонного стола должна располагаться с южной и юго-восточной стороны кухни; она должна быть освещена большими окнами, чтобы солнце щедро проникало внутрь и заливало кухню желтым светом — и утром и вечером.



с. 558, 562 нов [116](#) Многоуровневая крыша и [117](#) Защитная функция крыши, вы уже примерно представляете себе форму крыш и точно знаете, где нужна плоская крыша для устройства садов на крыше, прилегающих к помещениям на разных этажах, — см. шаблон [118](#) Сады на крыше. Данный шаблон поможет вам превратить любой черновой план в детальный проект кровли здания, с помощью которого будут реализованы вышеупомянутые шаблоны.

209 Проект кровли •



Какое устройство кровли будет органично сочетаться с характером дома?

с. 836 Исходя из аргументации, представленной в шаблоне [191](#) Геометрия внутреннего пространства, мы знаем, что большинство помещений в «органическом» доме будут иметь более или менее прямые стены (не обязательно идеально прямые), поскольку только в этом случае по обеим сторонам стен образуется позитивное пространство выпуклой формы.

Благодаря тем же доводам мы знаем, что большинство углов в здании будут более или менее прямыми (опять же не идеально прямыми), то есть в диапазоне от восьмидесяти до ста градусов.

Итак, мы знаем, что естественные формы помещений могут быть весьма разнообразными: полукруг, октагон и так далее, но по большей части это будут прямоугольники неидеальной формы.

с. 562 Также нам известно из шаблона [117](#) Защитная функция крыши, что каждое крыло по возможности должно целиком находиться под одной крышей, а в целом для здания требуется комбинация из плоских и наклонных или куполообразных крыш, причем плоских участков должно быть меньше.

с. 562, 563 Исходя из этого можно следующим образом сформулировать задачу, встающую при разработке проекта кровли: *как, имея весьма произвольный план описанного выше типа, обеспечить компоновку крыши, которая будет соответствовать рекомендациям шаблонов [116](#) Многоуровневая крыша, [117](#) Защитная функция крыши и [118](#) Сады на крыше?*

Прежде чем приступить к описанию процесса детального проектирования кровли, хотим обратить внимание на пять исходных положений, лежащих в его основе.

с. 972 1. Наклонные крыши могут действительно иметь скаты, также это могут быть своды цилиндрической формы или с криволинейными скатами, как описано в шаблоне [220](#) Сводчатые крыши. Общий алгоритм проектирования кровли во всех трех случаях одинаков (для сводов с криволинейными скатами уклон определяется как соотношение между их высотой и шириной).



«Скат» сводчатой крыши

2. Предположим, что все участки крыши, которые не являются плоскими, имеют примерно одинаковый уклон. Как правило, для определенной конструкции кровли в определенном климате лучше иметь одинаковый уклон на всех ее участках; к тому же это существенно упрощает строительство.



Одинаковый уклон на всех участках крыши

3. Поскольку все крыши имеют одинаковый уклон, то гребни крыш над самыми широкими крыльями и (или) самыми большими помещениями будут находиться на наибольшей высоте в сравнении с гребнями крыш над помещениями и крыльями зданий меньшей площади. Это согласуется с положени-

Любой паттерн описывает задачу,
которая снова и снова возникает
в нашей работе, а также
принцип её решения, причём таким
образом, что это решение можно
использовать много раз, ничего
не изобретая заново

Кристофер Александр (1977)

Паттерны описывают
взаимодействие объектов и классов,
адаптированных для решения общей
задачи проектирования
в конкретном контексте

ПРАВИЛО 1



Программируйте
в соответствии
с интерфейсом,
а не реализацией


```
interface IEmployee
{
    public function getSalary();
}

class Worker implements IEmployee
{
    private $salary;
    private $workHours;

    public function getSalary() { }
    private function getWorkDays() { }
}
```

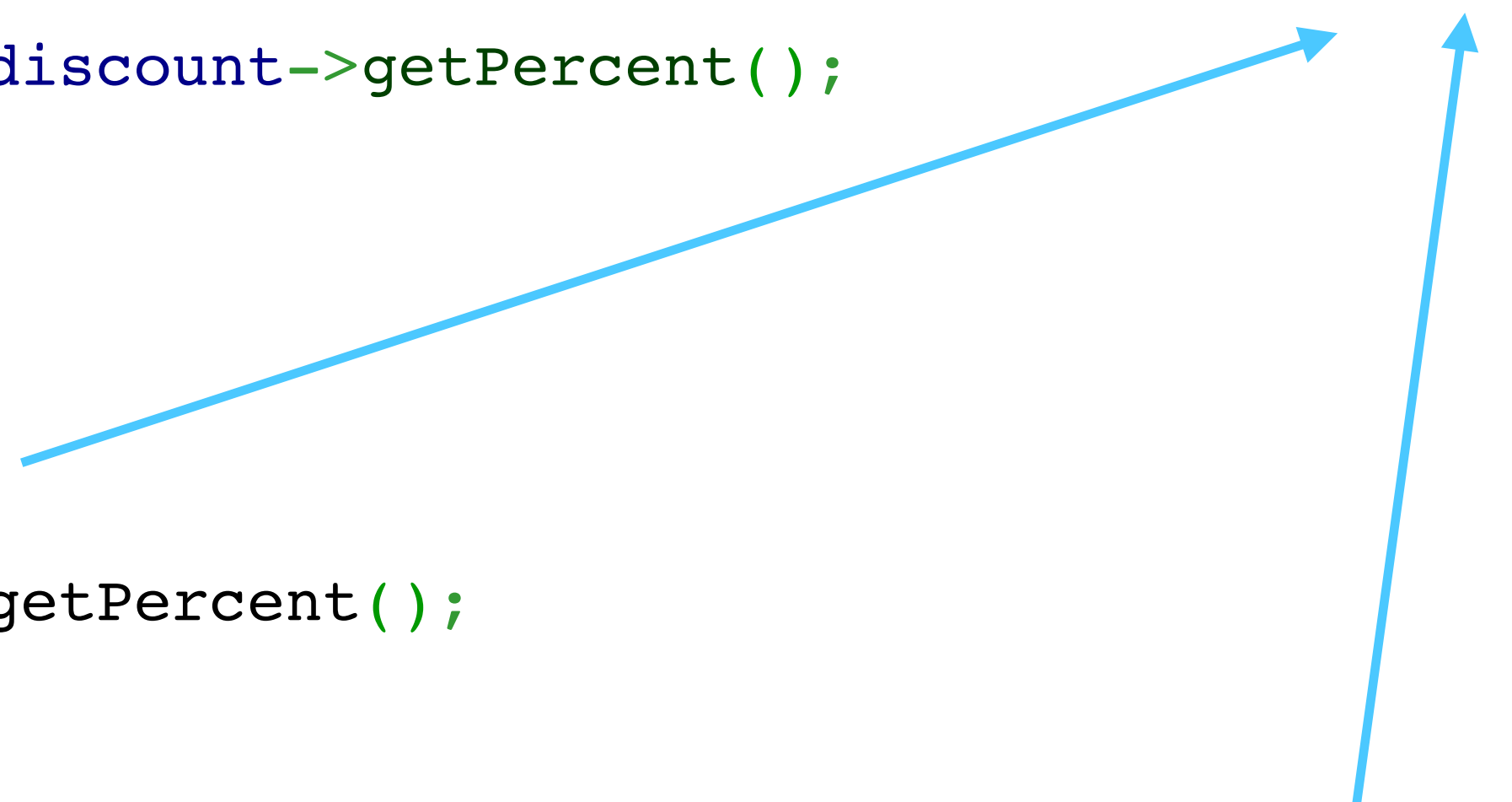

Характеристика интерфейсов

- Внешнее описание способов (методов) взаимодействия с классом
- Ничего не говорят о реализации класса
- Позволяют подставлять (использовать) разные объекты с одним интерфейсом
- Два класса с одним интерфейсом могут реализовывать разные действия


```
class Order
{
    public function getTotalAmount(IProduct $product, IDiscount $discount)
    {
        $percent = $discount->getPercent();
    }
}
```

```
interface IDiscount
{
    public function getPercent();
}
```

```
(new Order())->getTotalAmount(new Product(), new SuperVIPDiscount());
```



Плюсы

- Клиентский класс ничего не знает о получаемом объекте
- Клиентский класс оперирует исключительно описанием интерфейса
- Уменьшает зависимость классов и удовлетворяет Low Coupling


```
abstract class Employee
{
    abstract public function getWorkHours();

    public function calculateMonthSalary()
    {
        $workHours = $this->getWorkHours();
    }
}

class Worker extends Employee
{
    public function getWorkHours() { }
}
```


Характеристика абстрактных классов

- Игруют роль шаблона или скелета
- Задают базовое поведение для всех своих наследников
- Делегирует все или часть своих операций подклассам

Цель наследования

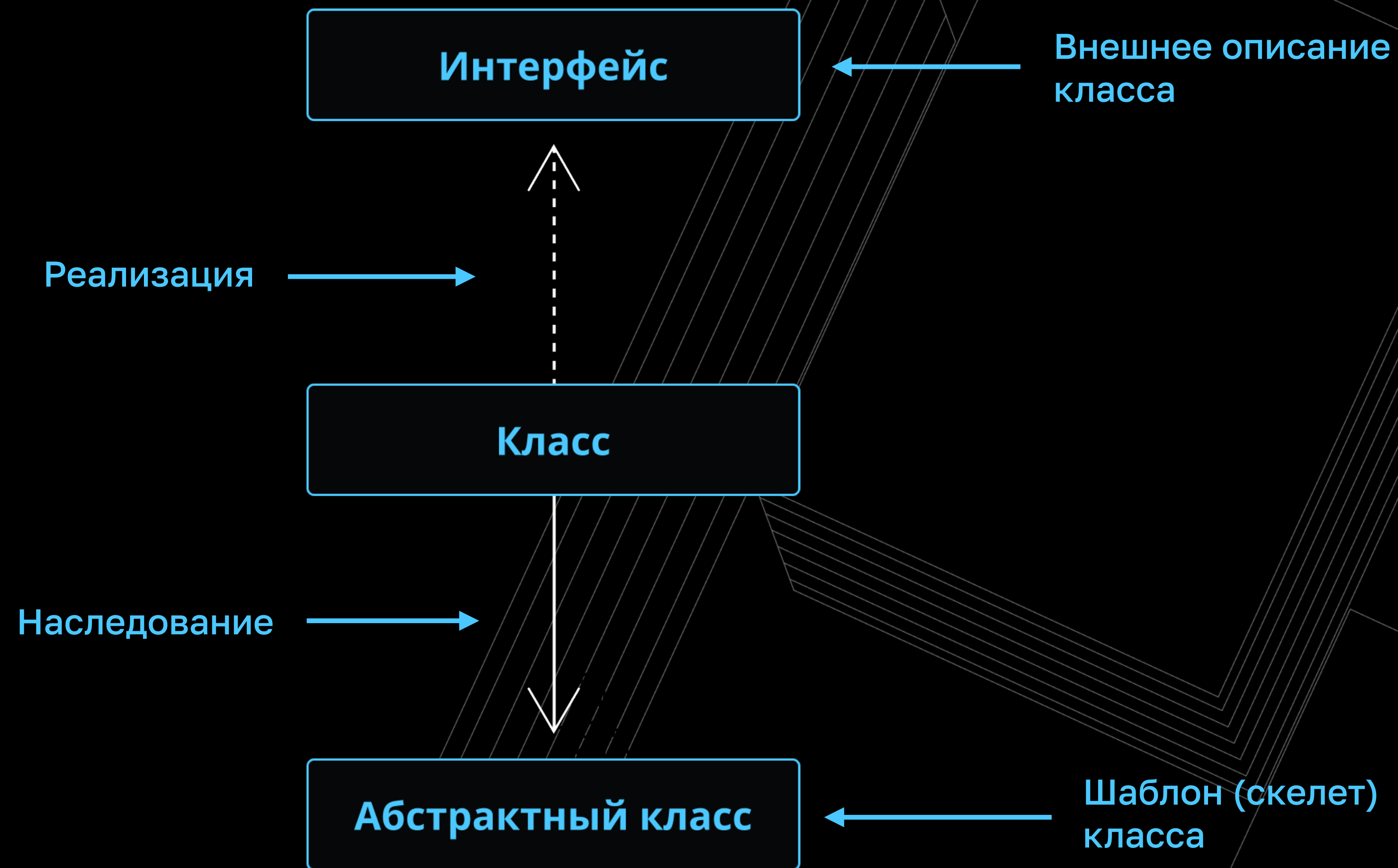
- Реализация базовых методов исходного класса (полиморфизм)
- Расширение функциональности класса за счёт переиспользования методов родительского класса


```
interface IEmployee
{
    public function getSalary();
}

abstract class Employee
{
    abstract public function getWorkHours();

    public function calculateMonthSalary()
    {
        $workHours = $this->getWorkHours();
    }
}

class Worker extends Employee implements IEmployee
{
    public function getSalary() { }
    public function getWorkHours() { }
}
```

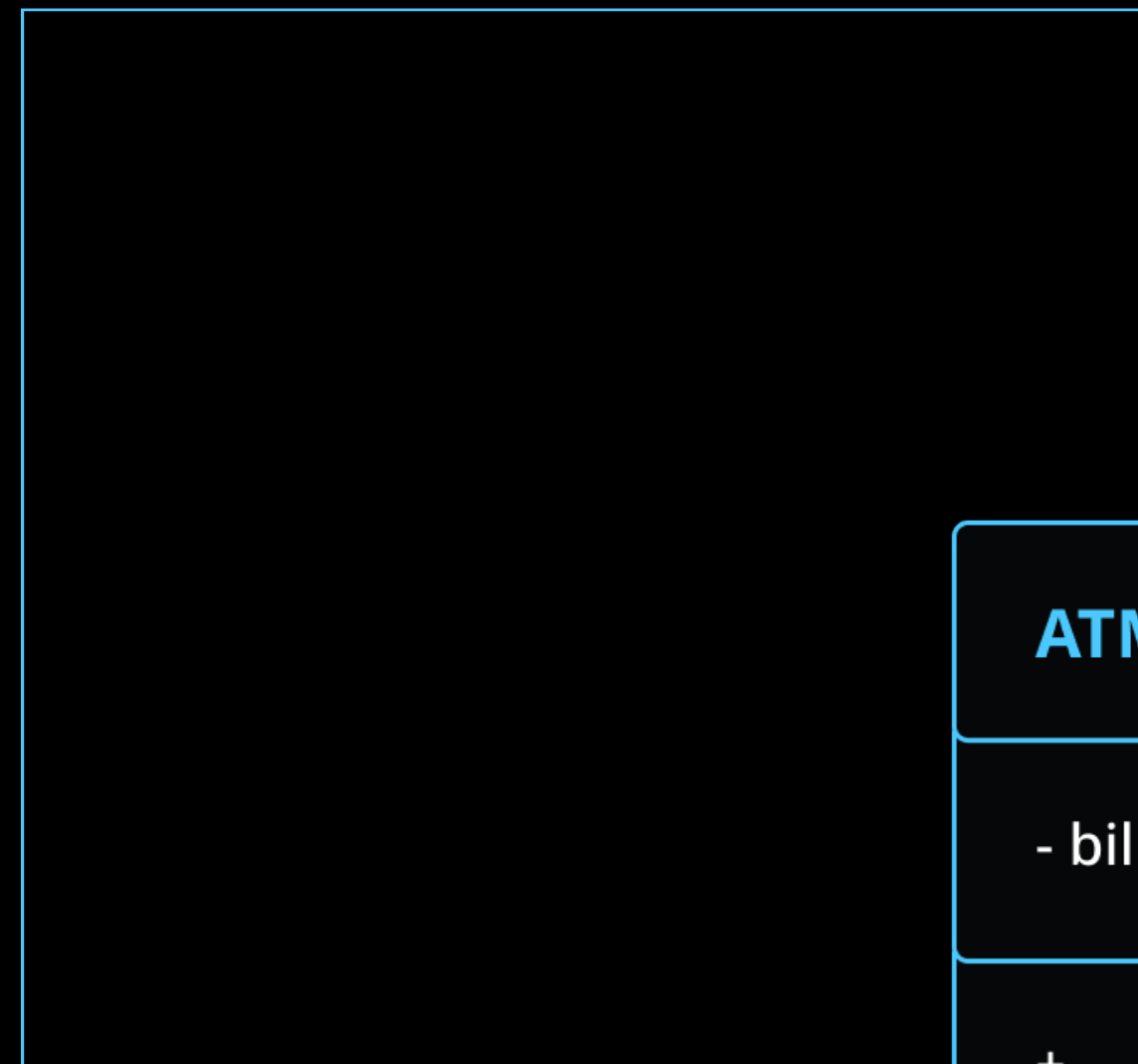
Используя класс,
ссылайтесь
на интерфейс или
абстрактный класс,
но не на него самого

ПРАВИЛО 2

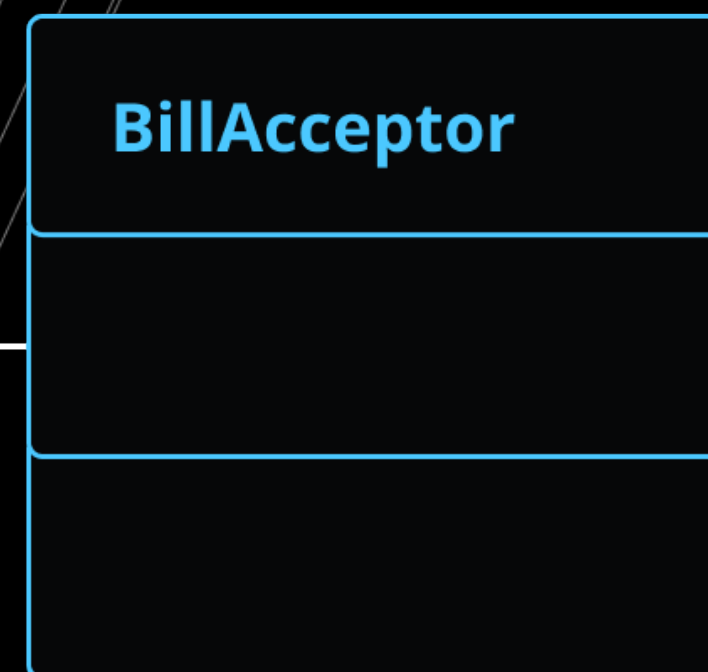


Предпочитайте композицию
наследованию класса

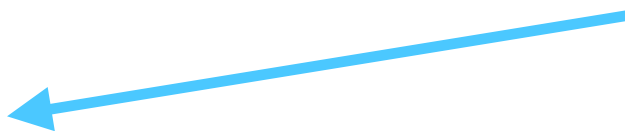
Контейнер



Объект



Композиция



```
$atm = new ATM(new BillAccepter());
```

```
class BillAccepter implements IAccepter
{
    // ..методы...
}
```

```
class ATM
{
    private $acceptor;

    public function __construct(IAcceptor $acceptor)
    {
        $this->acceptor = $acceptor;
    }
}
```


← Агрегация

```
$billAcceptor = new BillAcceptor();  
$atm = new ATM($billAcceptor);
```

```
class BillAcceptor implements IAcceptor  
{  
    // ..методы..  
}
```

```
class ATM  
{  
    private $acceptor;  
  
    public function __construct(IAcceptor $acceptor)  
    {  
        $this->acceptor = $acceptor;  
    }  
}
```


Композиция

Наследование

- Внутреннее устройство родительского класса известно подклассам
- Повторное использование за счёт порождения наследника

- Детали внутреннего устройства класса остаются скрытыми
- Повторное использование за счёт объединения классов

Композиция

Наследование

- Статический вид связи, создаётся на этапе компиляции
- Сильная связь между классами (изменение работы родителя влечёт к изменению всех наследников)

- Динамический вид связи, подменить класс возможно в runtime
- Объединение классов возможно за счёт хорошо спроектированных интерфейсов

Плюсы КОМПОЗИЦИИ

- Не нарушается инкапсуляция классов
- Все классы остаются небольшими и не раздуваются
- Каждый класс реализует свою небольшую задачу
- Классы связаны через интерфейс

Для улучшения
качества кода
отдавайте
предпочтение
композиции
объектов

avito.tech

Смотрите все серии на сайте
<https://avito.tech/patterns>

